CLOUD SECURITY

PBL — Complete Practical Workbook

Step-by-step AWS console instructions • commands • verification • screenshots • security controls • cleanup

Student: Ghanish Patil **PRN suffix used in examples:** 1047
Important: Replace example values (region, IP addresses, key-pair names, resource IDs) with your own values. Do not publish access keys, private keys, passwords, public IPs, or account IDs in screenshots.

This workbook follows the six practicals in the supplied Cloud Security PBL sheet. It adds operational steps and safe verification commands so each practical has a clear setup, test, evidence, and cleanup path.

Use only your own AWS account and instructor-authorized lab resources. Charges may accrue while EC2 instances, NAT gateways, load balancers, and storage remain active.

How to use this workbook

Read the safety notes first, then complete one practical at a time. Keep a small evidence folder containing screenshots, command output, and a short observation for every test. Console labels may vary slightly by AWS region or UI update.

Common conventions

Item	Example / meaning
AWS Region	Choose one region for all resources, e.g. ap-south-1 (Mumbai), if available to your account.
Linux example	Amazon Linux 2023; commands use sudo and systemd where appropriate.
Windows example	Windows Server EC2; connect using RDP and use PowerShell as Administrator.
PRN suffix	1047 is used in the page content examples. Change it if your assigned suffix differs.
Placeholder	<YOUR_PUBLIC_IP>, <INSTANCE_ID>, <BUCKET_NAME> mean replace with real values
Private IP examples	10.0.1.10 / 10.0.2.20 are examples only; use the addresses shown in your VPC.

Evidence checklist for every practical

- Screenshot of the AWS configuration page with the important settings visible.
- Screenshot of the resource state (instance, VPC, trail, policy, etc.).
- Terminal/PowerShell output proving the test passed or showing the error investigated.
- A brief note: expected result, actual result, and what it demonstrates.
- A security risks and mitigations paragraph.
- Cleanup confirmation showing that temporary resources were stopped or deleted.

Cost and security rule: Do not leave lab resources running after evidence collection. Never use root-account access keys in scripts. Use least privilege and only open inbound ports that the practical requires.

Practical 1 — Public Linux EC2 web server

Objective: Launch a Linux EC2 instance, install a web server, display a page containing the last four PRN digits (1047 in this example), and verify access over HTTP.

A. Launch the instance

- Open AWS Console → EC2 → Instances → Launch instances.
- Name: PBL1-Linux-Web. Select Amazon Linux 2023 AMI (or the instructor-approved Linux image).
- Choose an eligible instance type, such as t2.micro or t3.micro if available under your account limits.
- Select or create a key pair and download the private key once. Store it securely; do not upload it with the report.
- Network settings: select a lab VPC/default VPC, enable auto-assign public IP, and create/select a security group.
- Inbound rules: SSH TCP 22 from My IP only; HTTP TCP 80 from 0.0.0.0/0 for the public website demo. Do not expose SSH to 0.0.0.0/0.
- Launch the instance and wait until Instance state is Running and status checks pass.

B. Connect over SSH

On Windows, use PowerShell. First set the downloaded key path and replace the public DNS name with the one shown under EC2 → Instance details.

```
# PowerShell: move to the folder containing your .pem file
cd "$HOME\Downloads"

# Restrict key file permissions if SSH complains (run in PowerShell)
icacls .\pbl-key.pem /inheritance:r
icacls .\pbl-key.pem /grant:r "$($env:USERNAME):(R)"

# Connect (replace the host and key filename)
ssh -i .\pbl-key.pem ec2-user@<PUBLIC_DNS_OR_IP>
```

If using macOS/Linux, use chmod 400 on the private key before connecting. The default username for Amazon Linux is usually ec2-user; Ubuntu AMIs usually use ubuntu.

```
# macOS/Linux example
chmod 400 ./pbl-key.pem
ssh -i ./pbl-key.pem ec2-user@<PUBLIC_DNS_OR_IP>
```

C. Install Apache and publish the PRN page

```
# Run inside the Linux EC2 SSH session
sudo dnf update -y
sudo dnf install -y httpd
sudo systemctl enable --now httpd

# Create a simple page showing the PRN suffix
echo '<!doctype html><html><head><title>Cloud Security PBL</title></head><body><h1>Cloud Security PBL</h1><p>PRN su

# Verify service state and listening port
sudo systemctl status httpd --no-pager
sudo ss -lntp | grep ':80'
curl -I http://127.0.0.1/
curl http://127.0.0.1/
```

D. Verify from your own computer

```
# Run on your computer, not inside EC2
curl -I http://<PUBLIC_IP_OR_DNS>/
curl http://<PUBLIC_IP_OR_DNS>/
```

Open http:/// in a browser. Expected result: the page loads and visibly shows PRN suffix 1047. If it fails, check the instance is running, the public IP exists, the security group allows TCP 80, the route table has an Internet Gateway route, and httpd is active.

E. Troubleshooting checks

```
# On Linux EC2
sudo systemctl is-active httpd
sudo journalctl -u httpd --no-pager -n 50
sudo ss -lntp
curl -v http://127.0.0.1/

# From your computer (PowerShell)
Test-NetConnection <PUBLIC_IP> -Port 80
```

F. Risks, controls, and cleanup

- Risk: public HTTP is unencrypted. Control: use only a demo page with no secrets; production sites should use HTTPS.
- Risk: SSH exposure. Control: restrict TCP 22 to My IP and remove the rule when finished.
- Risk: forgotten running resources. Control: terminate the instance after collecting screenshots.
- Evidence: instance details, security group inbound rules, SSH terminal, systemctl status, browser page showing 1047, and cleanup state.

Practical 2 — Public Windows EC2 web server

Objective: Launch a Windows EC2 instance, configure a web server, publish a page showing PRN suffix 1047, and verify public HTTP access.

A. Launch Windows EC2

- EC2 → Instances → Launch instances. Name: PBL2-Windows-Web.
- Choose the instructor-approved Windows Server AMI.
- Choose an eligible instance type and create/select a key pair. Keep the .pem file secure; it is needed to decrypt the initial Windows password.
- Enable a public IP for this lab instance.
- Security group inbound rules: RDP TCP 3389 from My IP only; HTTP TCP 80 from 0.0.0.0/0 for the website demo.
- Launch and wait for both status checks to pass.

B. Connect with Remote Desktop

- Select the instance → Connect → RDP client.
- Choose Get password, upload/select your key pair private key, and decrypt the Administrator password.
- Open Remote Desktop Connection (mstsc.exe), enter the instance public IP/DNS, and sign in as Administrator.
- Do not include the decrypted password in screenshots or the report.

C. Install IIS and create the page

Open PowerShell as Administrator on the Windows EC2 instance.

```
# Install IIS web server and management tools
Install-WindowsFeature -Name Web-Server -IncludeManagementTools

# Create a simple HTML page
$html = @'
<!doctype html>
<html>
<head><title>Cloud Security PBL</title></head>
<body>
<h1>Cloud Security PBL</h1>
<p>PRN suffix: 1047</p>
<p>Windows EC2 web server (IIS)</p>
</body>
</html>
'@

Set-Content -Path 'C:\inetpub\wwwroot\index.html' -Value $html -Encoding UTF8

# Confirm IIS service and port 80 listener
Get-Service W3SVC
Start-Service W3SVC
Get-NetTCPConnection -LocalPort 80 -State Listen
```

D. Test locally on Windows

```
# On the Windows EC2 instance
Invoke-WebRequest http://localhost/ -UseBasicParsing
curl.exe -I http://127.0.0.1/
```

E. Test from your computer

```
# On your local Windows PowerShell
Test-NetConnection <WINDOWS_EC2_PUBLIC_IP> -Port 80
Invoke-WebRequest http://<WINDOWS_EC2_PUBLIC_IP>/ -UseBasicParsing
```

Expected result: HTTP succeeds and the returned page contains “PRN suffix: 1047”. Open http:/// in a browser as a second proof.

F. Troubleshooting

```
# On the Windows instance, elevated PowerShell
Get-Service W3SVC
Get-NetFirewallProfile
Get-NetTCPConnection -LocalPort 80 -State Listen
Get-Content C:\inetpub\wwwroot\index.html

# If needed, confirm IIS feature state
Get-WindowsFeature Web-Server
```

- If local HTTP works but external HTTP fails, inspect the EC2 security group, public IP, VPC route table/Internet Gateway, and Windows Defender Firewall.
- Keep RDP TCP 3389 restricted to My IP. Never expose RDP to the entire internet for convenience.
- Evidence: Windows instance details, security group, RDP session (without password), IIS feature/service, local test, public browser result.
- Cleanup: terminate the Windows instance after collecting evidence; Windows instances can incur meaningful hourly charges.

Practical 3 — Isolated attacker/target lab and connectivity testing

Scope and safety: Perform this only in a lab you own or your instructor explicitly authorizes. Use disposable instances, private networking, and no real user data. This workbook includes safe connectivity checks. For the actual reverse-shell demonstration, use only the instructor-provided lab procedure/payload in the authorized isolated environment; this workbook does not provide a reusable reverse-shell payload.

Objective: Build an isolated two-instance lab, restrict SSH to your public IP, test IP reachability and TCP service connectivity, collect logs, and document the reverse-shell demonstration only within the approved lab exercise.

A. Recommended isolated layout

Component	Example	Required control
Lab VPC	10.20.0.0/16	Separate from production and personal networks.
Private subnet	10.20.1.0/24	Use private IPs; no direct public IP on target.
Attacker/test VM A	10.20.1.10	Authorized security testing VM; administrative access restricted.
Disposable target VM B	10.20.1.20	Only instructor-approved target; snapshot first.
Security group	Lab-only SG	Allow only required ports from the other lab SG or exact lab IP.

A fully private lab is preferred. If you must administer a VM from the internet, use a controlled jump host/VPN or temporarily restrict SSH/RDP to your exact public IP. Do not assign a public IP to the target merely to make testing easier.

B. Configure networking and access

- Create a dedicated VPC and private subnet, or use the instructor-provided isolated lab VPC.
- Launch VM A and VM B in the same private VPC/subnet unless the lab instructions require otherwise.
- Record each instance's private IP from EC2 → Instances → Networking.
- Security groups: permit only the exact test ports needed, preferably referencing the peer security group rather than opening broad CIDR ranges.
- For administrative SSH, allow TCP 22 from your current public IP only, and remove the rule after administration. If the instances are private-only, use the approved VPN/jump-host/SSM path.
- Do not allow inbound connections from arbitrary internet addresses to the target. Do not bridge the lab to campus or home networks.

C. Verify IP addresses and routes

Run these checks on both Linux VMs. Use the private IPs actually assigned by AWS; the example addresses below are illustrative.

```
# On both VM A and VM B
hostname
hostname -I
ip -brief address
ip route

# On VM A, substitute VM B's private IP
ip route get 10.20.1.20
ping -c 4 10.20.1.20
```

Interpretation: hostname/IP commands establish which VM you are on and its interfaces. ip route get shows the route the kernel would use. Ping may fail even when TCP works because ICMP can be blocked; a failed ping alone does not prove the hosts are disconnected.

D. Test a harmless TCP service from target to attacker / attacker to target

Use a benign temporary HTTP service to demonstrate that the network path and security-group rules work. Example below starts a basic Python HTTP server on VM B at TCP 8080. Do not use this service to host sensitive files.

```
# On VM B (the disposable target): create a harmless test directory
mkdir -p ~/pbl-connectivity
printf 'PBL connectivity test only\n' > ~/pbl-connectivity/index.html
cd ~/pbl-connectivity

# Bind to VM B's private interface if supported; otherwise use 0.0.0.0
python3 -m http.server 8080 --bind 0.0.0.0
```

Keep that terminal open. In the VM B security group, temporarily allow inbound TCP 8080 from VM A's security group (preferred) or VM A's exact private IP /32. Do not allow 8080 from 0.0.0.0/0.

```
# On VM B in a second terminal: confirm the listener
sudo ss -lntp | grep ':8080'

# On VM A: test TCP reachability
nc -vz 10.20.1.20 8080

# On VM A: make an HTTP request and inspect the response
curl -v --max-time 5 http://10.20.1.20:8080/
```

Expected result: nc reports a successful connection and curl prints “PBL connectivity test only”. This proves a TCP/HTTP path is working; it does **not** prove a reverse shell was established.

E. Optional port test from Windows PowerShell

```
# From a Windows lab machine with network access to the target
Test-NetConnection 10.20.1.20 -Port 8080
curl.exe -v --max-time 5 http://10.20.1.20:8080/
```

F. Troubleshoot a failed connection

```
# On VM A and VM B
ip -brief address
ip route
sudo ss -lntp
sudo journalctl -n 50 --no-pager

# On VM B, inspect firewall rules (commands depend on firewall tooling)
sudo systemctl status firewalld --no-pager
sudo firewall-cmd --list-all 2>/dev/null || true

# From VM A, retry with a short timeout
nc -vz -w 3 10.20.1.20 8080
curl -v --connect-timeout 3 --max-time 5 http://10.20.1.20:8080/
```

- Check the IP address and subnet: the target private IP must be correct.
- Check the service is running and listening on TCP 8080; a service bound only to 127.0.0.1 will not accept remote connections.
- Check both security groups and network ACLs. AWS security groups are stateful; network ACLs are stateless and may need return traffic permitted.
- Check host firewall rules on the target.
- Remember that ping uses ICMP while nc/curl use TCP; test the protocol actually required by the practical.

G. Reverse-shell part — how to document safely

For the assignment's reverse-shell portion, follow the instructor's exact lab handout and use only the supplied controlled exercise on the disposable target. Do not run reverse-shell code on a public server, a third-party machine, or a system containing real data.

- Before testing, record written scope: target instance ID/private IP, permitted test window, and permitted technique.
- Snapshot the disposable target and confirm it contains no important data or credentials.
- Use the instructor-provided test procedure rather than copying a payload from an untrusted source.
- Capture evidence from the approved listener/target terminal and relevant system logs, redacting tokens, secrets, and unrelated data.
- After the exercise, stop the test listener, remove temporary rules, revert or terminate the disposable target, and verify that no unexpected process or open port remains.

H. Logs and post-test checks

```
# Linux: recent system log entries (service names vary by distribution)
sudo journalctl --since "15 minutes ago" --no-pager

# Check listeners after the test
sudo ss -lntp

# Stop the benign Python HTTP server with Ctrl+C in its terminal.
# Then confirm port 8080 is no longer listening:
sudo ss -lntp | grep ':8080' || echo "Port 8080 is no longer listening"
```

Evidence: VPC/subnet layout, private IPs, restrictive security-group rules, successful/failed nc and curl tests, relevant logs, approved lab demonstration evidence, and proof of cleanup. Clearly label all addresses and data as lab-only.

Practical 4 — Public and private subnets; public server retrieves private website

Objective: Create a VPC with public and private subnets, place a public EC2 server in the public subnet and a web server in the private subnet, then use curl from the public server to retrieve the private server's static page. The private server must not be directly accessible from the internet.

A. Create the VPC and subnets

- VPC Console → Your VPCs → Create VPC. Example IPv4 CIDR: 10.30.0.0/16.
- Create a public subnet such as 10.30.1.0/24 and a private subnet such as 10.30.2.0/24 in the same Availability Zone for a simple lab.
- Create and attach an Internet Gateway (IGW) to the VPC.
- Public route table: associate the public subnet and add 0.0.0.0/0 → Internet Gateway.
- Private route table: associate the private subnet. Do not add a direct Internet Gateway route.
- For this practical's minimum design, the private web server needs no outbound internet access. If software downloads are required, use a controlled NAT design only if permitted and remember NAT gateways incur charges.

B. Launch two Linux EC2 instances

Instance	Subnet	Public IP	Security group
PBL4-Public	Public subnet	Yes	SSH 22 from My IP; HTTP 80 only if needed; allow outb
PBL4-Private-Web	Private subnet	No	HTTP 80 inbound only from public server's security grou

Use Amazon Linux 2023 for both. If you cannot administer the private instance, use AWS Systems Manager Session Manager with the required IAM role and network endpoints, or use the instructor-approved jump-host approach. Do not temporarily make the private instance public just for convenience.

C. Install web service on private instance

On the private server, use an approved administration path. If it has no outbound route, package installation may not work unless the AMI already includes the package or the lab provides an approved repository path. If installation is possible:

```
sudo dnf install -y httpd
sudo systemctl enable --now httpd
echo '<h1>Private subnet website</h1><p>Retrieved only from inside the VPC.</p>' | sudo tee /var/www/html/index.htm
sudo systemctl status httpd --no-pager
sudo ss -lntp | grep ':80'
curl -I http://127.0.0.1/
```

D. Test from the public server

```
# On the public EC2 instance; replace with private server's private IP
curl -v --connect-timeout 5 http://10.30.2.10/
curl -I http://10.30.2.10/
```

Expected result: curl returns the private page. The public server reaches the private server using its private IP over the VPC network.

E. Prove the private server is not internet-accessible

- In EC2 details, verify the private server has no public IPv4 address.

- Verify the private subnet route table has no 0.0.0.0/0 route to an Internet Gateway.
- Verify the private server security group allows TCP 80 only from the public server's security group.
- From your own computer, do not expect direct HTTP access to the private IP; RFC1918 private IPs are not internet-routable.
- Capture the route table, subnet association, private instance networking tab, and successful curl from the public instance.

F. Troubleshooting

```
# On both instances
ip -brief address
ip route
sudo ss -lntp

# From public instance to private server
nc -vz -w 3 10.30.2.10 80
curl -v --connect-timeout 5 http://10.30.2.10/
```

If the request fails, check the private server service, security group source, network ACLs, subnet associations, and that both instances are in the intended VPC. A private subnet without NAT/egress endpoints may be unable to download packages; that does not prevent private VPC traffic.

G. Security and cleanup

- The public server is a potential entry point; patch it and restrict administrative access.
- The private server should not have a public IP and should accept traffic only from approved internal sources.
- Delete both EC2 instances and lab-only VPC resources after evidence collection. Remove NAT gateways if created, then release unused Elastic IPs.

Practical 5 — CloudTrail management events delivered to S3

Objective: Configure CloudTrail to record management events and deliver log files to an S3 bucket; perform sample EC2 and IAM management actions; then verify that logs arrived and contain the expected events.

A. Create a dedicated S3 bucket

- S3 → Create bucket. Use a globally unique name such as pbl-cloudtrail-.
- Keep Block Public Access enabled. Do not make the bucket or objects public.
- Enable default encryption (SSE-S3 or an instructor-approved KMS key).
- Enable bucket versioning if required by your lab rubric.
- Use a dedicated bucket for this exercise and do not store unrelated files in it.

B. Create a CloudTrail trail

- CloudTrail → Trails → Create trail.
- Trail name: PBL-Management-Trail. Choose the trail to apply to all regions if allowed by the assignment.
- Choose the dedicated S3 bucket created above.
- For event type, enable Management events and include Read and Write management events so actions such as RunInstances, TerminateInstances, and CreateUser can be captured.
- Create the trail and ensure logging is enabled.

C. Generate the three required management actions

Use a lab-only EC2 instance and a disposable IAM test user. Creating or deleting IAM users requires permission and may be restricted by an organization; if blocked, capture the denied result and ask the instructor for the approved account.

- EC2 creation: launch a small temporary EC2 instance named PBL5-Trail-Test. Record its instance ID and creation time.
- EC2 termination: terminate that same test instance after recording its ID. Do not terminate any unrelated instance.
- IAM user creation: IAM → Users → Create user. Use a clearly labeled disposable name such as pbl-cloudtrail-test-user. Do not create access keys or grant permissions unless the rubric explicitly requires it.
- If allowed, delete the disposable IAM user after confirming the event is logged. Note that deleting a user may require removing attached policies/keys first.

D. Find and inspect events

CloudTrail → Event history can show recent management events. Filter by event name and time. Expected event names commonly include RunInstances, TerminateInstances, and CreateUser.

Action	Likely event name	Evidence to record
Create EC2 instance	RunInstances	Event time, event name, user identity, instance ID/resources.
Terminate EC2 instance	TerminateInstances	Event time, event name, user identity, instance ID/resources.
Create IAM user	CreateUser	Event time, event name, user identity, target user name.

E. Verify delivery to S3

CloudTrail delivery can take several minutes. Open the selected S3 bucket and browse the AWSLogs//CloudTrail///// path. Do not share the account ID publicly.

```
# Optional AWS CLI checks, if AWS CLI is configured with an authorized role
aws sts get-caller-identity
aws cloudtrail get-trail-status --name PBL-Management-Trail
aws s3 ls s3://<BUCKET_NAME>/AWSLogs/ --recursive
```

Do not paste credentials into commands or screenshots. If the AWS CLI is not configured, use the console: CloudTrail Event history for event evidence and S3 object listing for delivery evidence.

F. What to write in the observation

- CloudTrail records management activity with event time, identity, action, request details, and source context where available.
- CloudTrail is an audit trail; it does not automatically prevent the action. Prevention is handled through IAM, SCPs, permissions boundaries, and related controls.
- S3 access controls, encryption, retention, and log-file integrity validation help protect audit evidence.

G. Cleanup

- Delete the disposable IAM user only after removing any resources/keys attached to it and confirming the event evidence.
- Terminate the temporary EC2 instance.
- Keep or delete the trail and bucket according to instructor requirements. If deleting, first stop logging/delete the trail as appropriate, then empty and delete the dedicated bucket. Do not delete shared organizational audit logs.

Practical 6 — Read-only cybersecurity analyst IAM policy

Objective: Create a custom read-only IAM policy for CloudTrail, CloudWatch, GuardDuty, AWS Config, and Security Hub; enable MFA; configure an account password policy; and verify that write actions are denied.

A. Create a least-privilege policy

IAM → Policies → Create policy → JSON. The example below grants common read/list/describe access to the requested services. AWS action names and service coverage can change; validate against the current IAM service authorization reference and your instructor's rubric before use. This is a starting point, not a guarantee that every console page will be fully populated.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadCloudTrail",
      "Effect": "Allow",
      "Action": [
        "cloudtrail:Describe*",
        "cloudtrail:Get*",
        "cloudtrail:List*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadCloudWatch",
      "Effect": "Allow",
      "Action": [
        "cloudwatch:Describe*",
        "cloudwatch:Get*",
        "cloudwatch:List*",
        "logs:Describe*",
        "logs:Get*",
        "logs:List*",
        "logs:FilterLogEvents",
        "logs:StartQuery",
        "logs:StopQuery",
        "logs:GetQueryResults"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadGuardDuty",
      "Effect": "Allow",
      "Action": [
        "guardduty:Get*",
        "guardduty:List*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadConfig",
      "Effect": "Allow",
      "Action": [
        "config:Describe*",
        "config:Get*",
        "config:List*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "ReadSecurityHub",
      "Effect": "Allow",
      "Action": [
        "securityhub:BatchGet*",
        "securityhub:Describe*",
```

```
        "securityhub:Get*",
        "securityhub:List*"
      ],
      "Resource": "*"
    }
  ]
}
```

The policy intentionally contains no Allow statements for create, update, delete, enable, disable, or other write operations. A separate explicit Deny is usually unnecessary for the basic demonstration, but another attached policy could grant extra permissions. For a strict read-only analyst identity, attach only the required policies and test effective permissions.

B. Create and attach the policy

- Save the policy with a clear name such as PBL-CybersecurityAnalyst-ReadOnly.
- Create a dedicated test user or role according to the instructor's instructions.
- Attach only the custom read-only policy and the minimum additional permissions required for sign-in/MFA management if applicable.
- Do not attach AdministratorAccess or broad PowerUser permissions to the analyst identity.
- For real environments, prefer IAM Identity Center/federated access and short-lived role sessions rather than long-lived IAM user credentials.

C. Enable MFA

For a lab IAM user: IAM → Users → select the test user → Security credentials → Assign MFA device. Follow the authenticator app or hardware MFA wizard. Save recovery information securely and never place QR codes, OTPs, or recovery codes in screenshots.

If the lab uses an IAM role or IAM Identity Center, follow the account's supported MFA configuration. Do not disable an existing MFA device or change production identity settings without authorization.

D. Configure an account password policy

IAM → Account settings → Password policy. Set a policy that meets the rubric, such as a suitable minimum length, complexity requirements, password reuse prevention, and expiration only if required by the organization's policy. Save the policy and capture the settings screen. Avoid imposing arbitrary expiration rules on real users without policy guidance.

E. Verify read actions and denied write actions

Use the test identity to open the five requested services and confirm that read-only pages load where data exists. Then test a harmless write attempt only in the lab. For example, attempt to create a CloudTrail trail or change a test CloudWatch alarm; do not modify production resources.

```
# Optional AWS CLI checks, run using the test analyst identity
aws sts get-caller-identity

# Read-only calls that should generally be permitted by the policy
aws cloudtrail describe-trails
aws cloudwatch describe-alarms
aws guardduty list-detectors
aws configservice describe-configuration-recorders
aws securityhub get-findings --max-results 1

# A write operation expected to fail with AccessDenied if no other policy grants it:
aws cloudtrail create-trail --name PBL-Should-Be-Denied --s3-bucket-name <BUCKET_NAME>
```

The last command is a negative permission test; it should return AccessDenied and should not create a trail. If the command succeeds, stop and inspect all policies attached to the test identity, group membership, permissions

boundaries, session policies, and organization controls. Clean up only if a resource was unexpectedly created and you are authorized to delete it.

F. Common limitations and troubleshooting

- A read-only policy cannot configure GuardDuty, Security Hub, AWS Config, or CloudTrail. Service setup may require a separate administrator to enable those services.
- Some console pages need supporting read permissions from dependent services; add narrowly scoped read actions only when a specific page requires them.
- The wildcard patterns above are convenient for a classroom exercise but can be broader than a production least-privilege policy. Refine actions and resource ARNs for production use.
- MFA and password-policy settings may be managed centrally or blocked by organizational controls; do not attempt to bypass them.

G. Evidence and cleanup

- Screenshots: policy JSON/name, policy attachment, MFA enabled status (no secret codes), password policy settings, successful read-only command outputs, and AccessDenied response for the write test.
- Delete the test identity and lab policy only if the instructor requires it and after saving evidence. Never remove an organization's real users or policies.

Final report format — copy this structure for each practical

Use the same report headings for all six practicals. Replace the sample wording with what you actually did and observed; do not claim a test passed unless your evidence confirms it.

1. Aim / Problem Statement

State the objective in 2–4 lines.

2. AWS Services Used

List EC2, VPC, IAM, CloudTrail, S3, CloudWatch, GuardDuty, Config, Security Hub, or other services actually used.

3. Architecture / Configuration

Describe region, VPC/subnet, instance OS, security-group rules, IAM policy, and relevant settings.

4. Procedure

Summarize the console steps and include the important commands executed.

5. Testing and Results

Use a table: test, expected result, actual result, pass/fail, evidence filename.

6. Security Risks and Mitigations

Discuss exposure, least privilege, logging, encryption, network isolation, and resource cleanup as applicable.

7. Screenshots / Evidence

Number each screenshot and add a one-line caption.

8. Conclusion

State what the practical demonstrated and any limitations.

9. Cleanup

List terminated instances, deleted temporary IAM users, removed temporary rules, and any retained resources.

Suggested test-result table

Test	Expected	Actual	Status	Evidence
Linux website	HTTP page displays PRN suffix	Fill in	Pass/Fail	P1-04.png
Windows website	IIS page displays PRN suffix	Fill in	Pass/Fail	P2-04.png
Lab connectivity	Authorized TCP/HTTP test succeeds	Fill in	Pass/Fail	P3-03.png
Private website	Public EC2 curl retrieves private page	Fill in	Pass/Fail	P4-04.png
CloudTrail	Required management events appear	Fill in	Pass/Fail	P5-03.png
IAM read-only	Read allowed; write attempt denied	Fill in	Pass/Fail	P6-05.png

Final cleanup checklist

- Terminate all temporary EC2 instances created for the practicals.
- Remove temporary inbound rules and lab-only security-group rules.

- Delete disposable IAM users and access keys created for the exercise, if applicable and authorized.
- Delete lab-only VPCs, subnets, route tables, Internet Gateways, NAT gateways, and unused Elastic IPs in the correct dependency order.
- Review S3 buckets, CloudTrail trails, snapshots, and logs. Retain only what the instructor requires; never delete shared audit data.
- Check EC2, VPC, S3, and billing/cost views for remaining chargeable resources.

Submission reminder: Include the problem and solution, configuration screenshots, command/test evidence, security risks and controls, conclusion, and cleanup confirmation. Redact credentials, passwords, private keys, MFA QR codes/OTPs, and sensitive account details.